
Informed Search

CS5491: Artificial Intelligence
JIANYUAN GUO, ZHICHAO LU

Content Credits: Prof. Wei's CS4486 Course
and Prof. Boddeti's AI Course



TODAY (PART 2)

Informed Search Methods

- Heuristics
- Greedy Search
- A^* Search



XKCD

Reading

- Today's Lecture: RN Chapter 3.5-3.6, 4.1-4.2



SEARCH PROBLEMS

Uninformed Search

Methods we saw:

- DFS
- BFS
- UCS

Searching with your **EYES CLOSED**

Informed Search

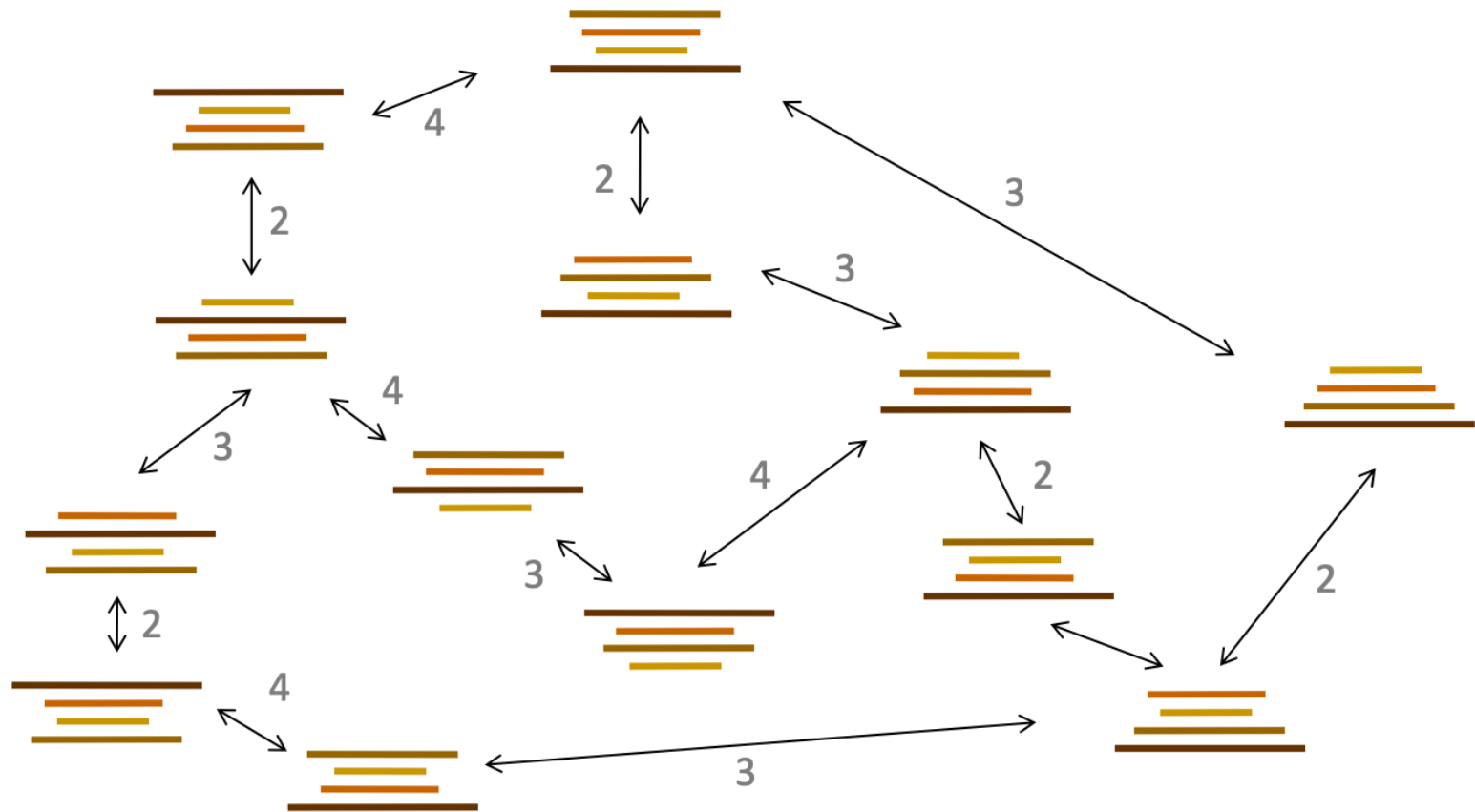
Methods we will see:

- Greedy
- A^*

Searching with your **EYES OPEN**



EXAMPLE: PANCAKE PROBLEM



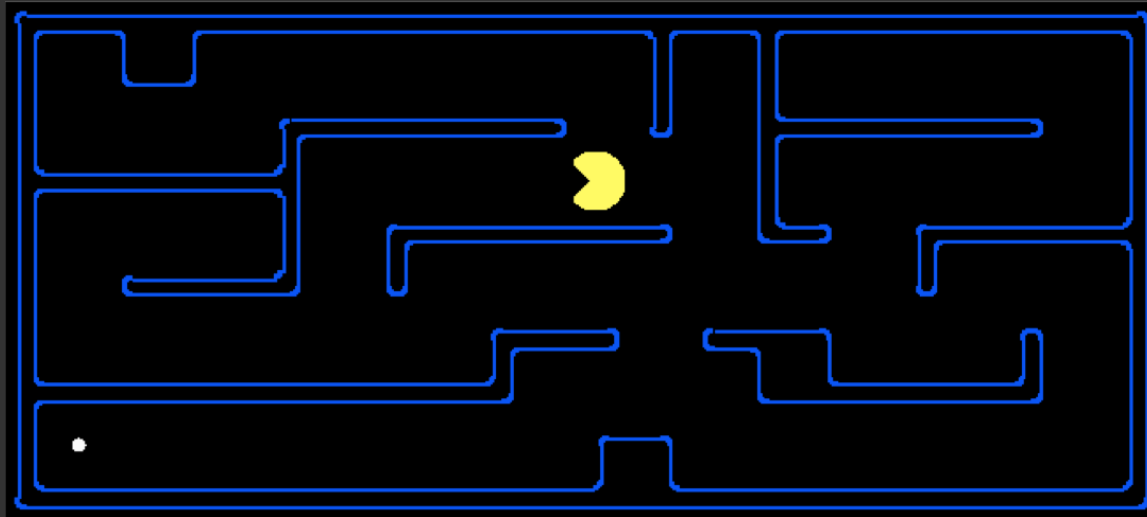
HEURISTICS



SEARCH HEURISTICS

A heuristic is:

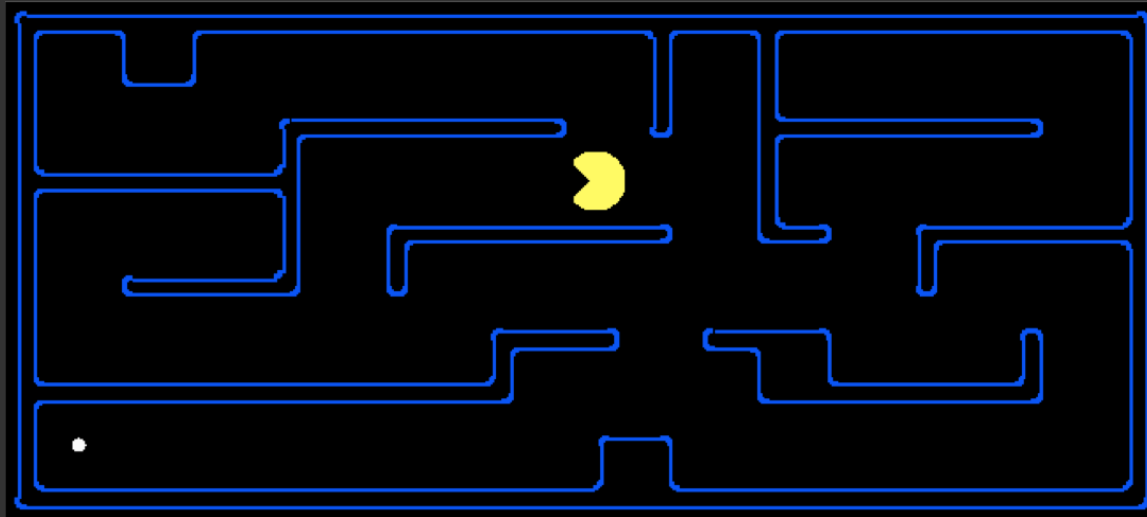
- A function that estimates how close a state is to a goal



SEARCH HEURISTICS

A heuristic is:

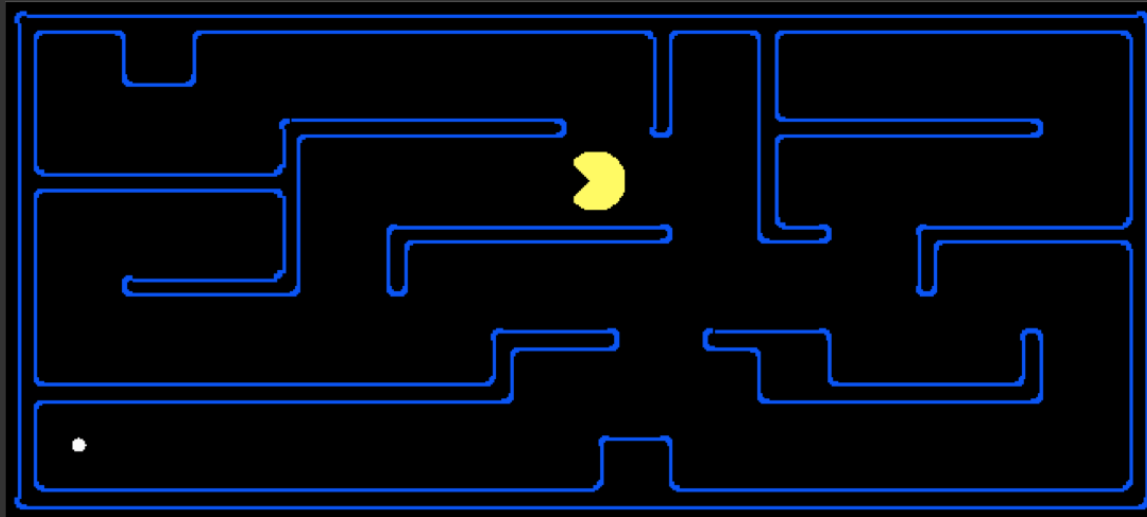
- A function that estimates how close a state is to a goal
- Designed for a particular search problem



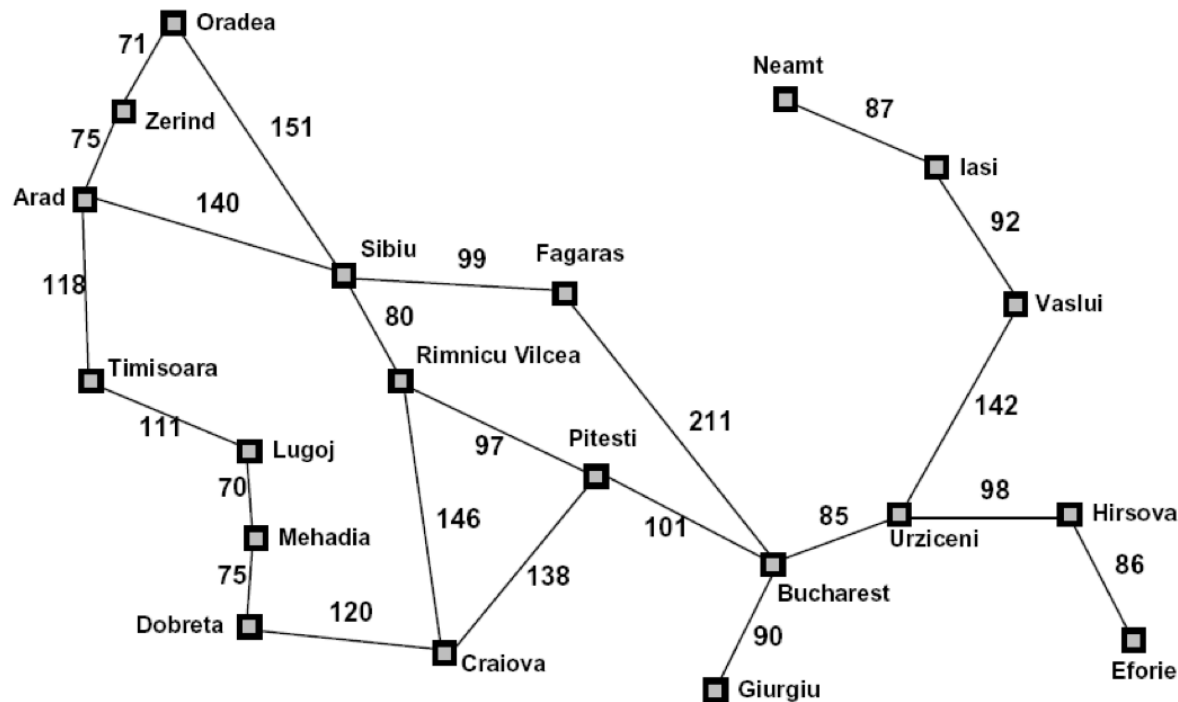
SEARCH HEURISTICS

A heuristic is:

- A function that estimates how close a state is to a goal
- Designed for a particular search problem
- Examples: Manhattan distance, Euclidean distance



EXAMPLE: HEURISTIC FUNCTION

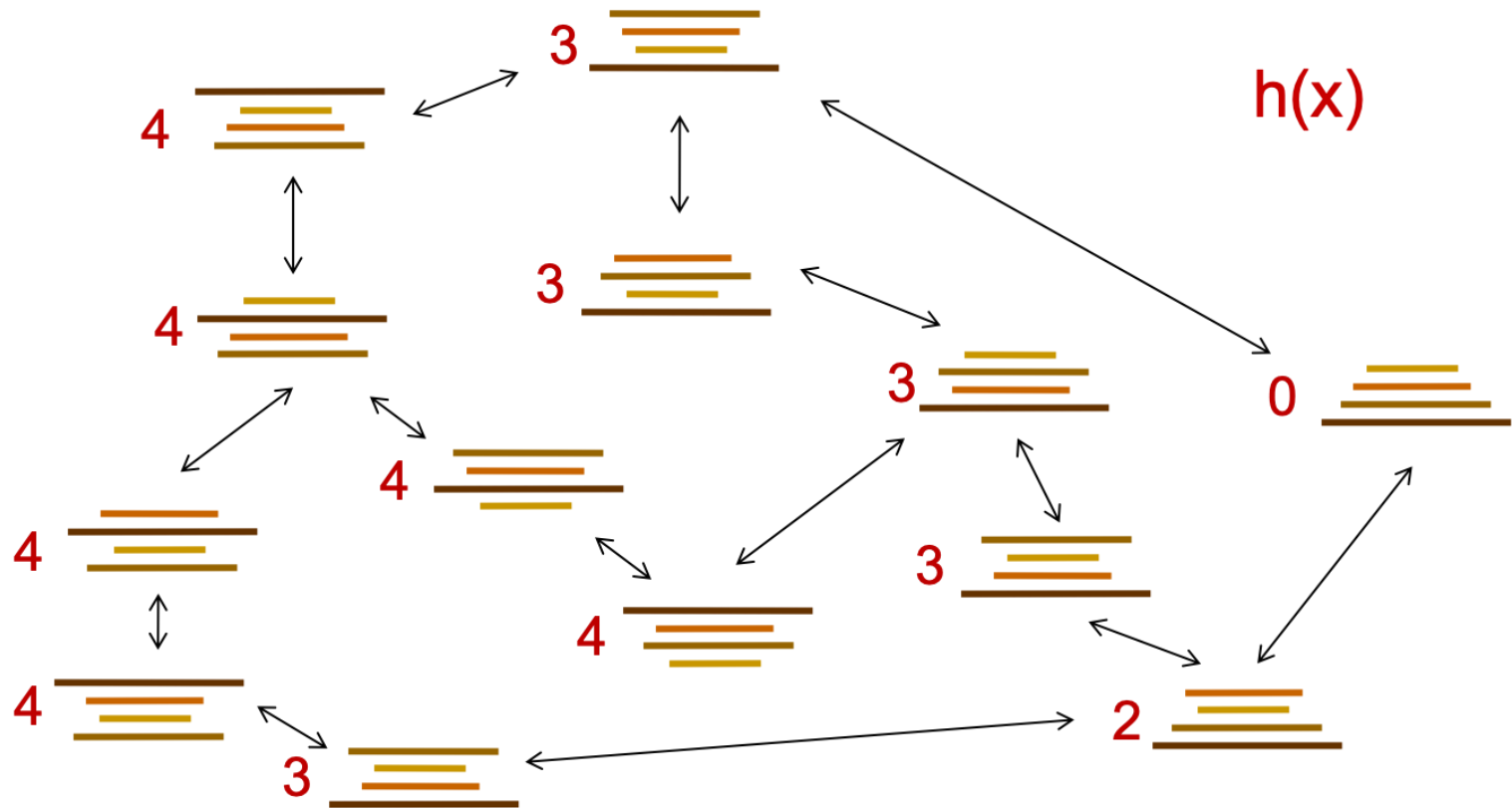


Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

EXAMPLE: HEURISTIC FUNCTION

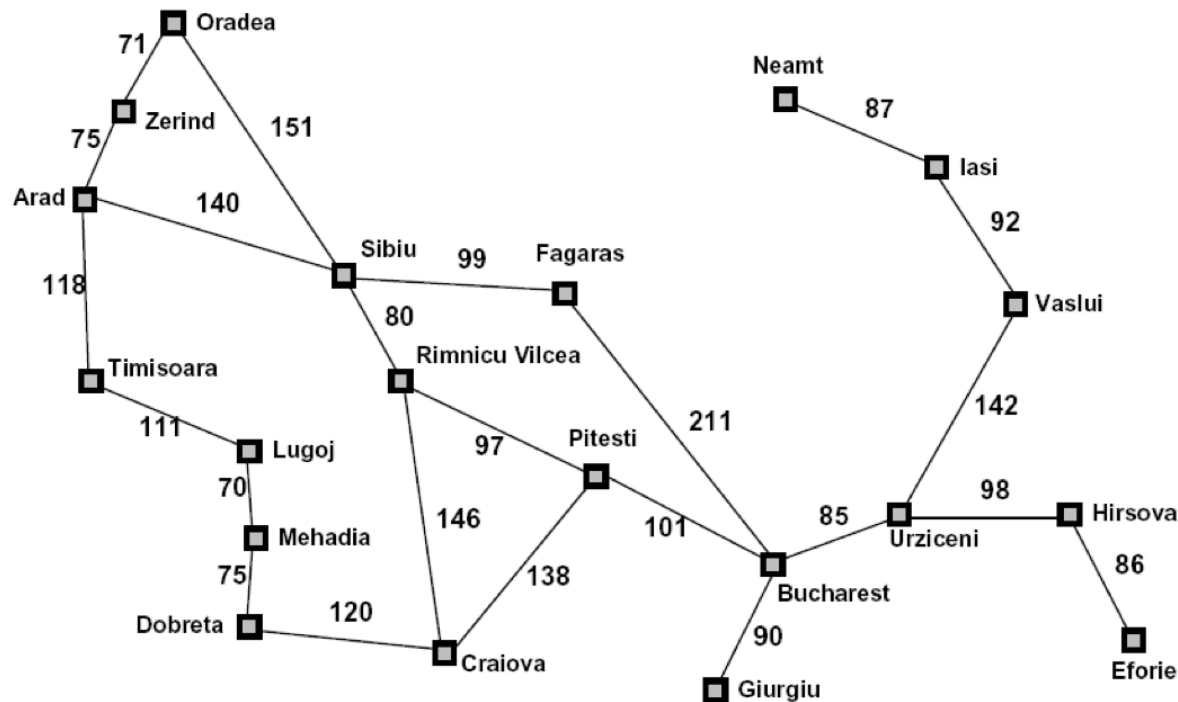
Eg: index of the largest pancake that is still out of place



GREEDY SEARCH



EXAMPLE: HEURISTIC FUNCTION



Straight-line distance
to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

GREEDY SEARCH: ROMANIA

Expand the node that seems closest...

Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374



GREEDY SEARCH: ROMANIA

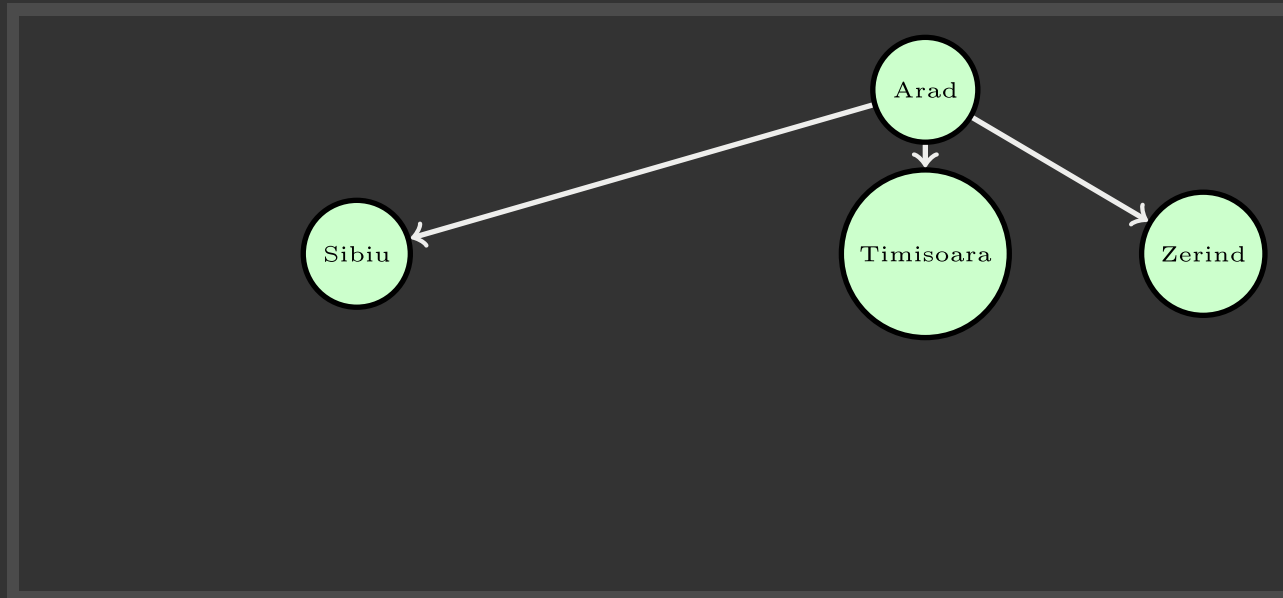
Expand the node that seems closest...



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

GREEDY SEARCH: ROMANIA

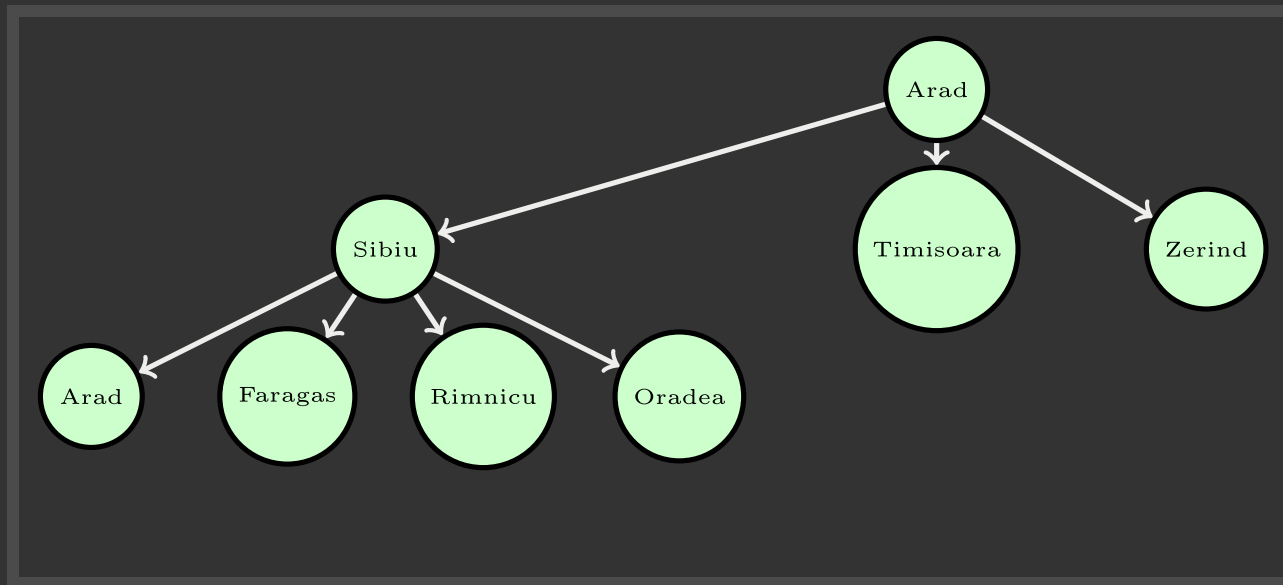
Expand the node that seems closest...



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

GREEDY SEARCH: ROMANIA

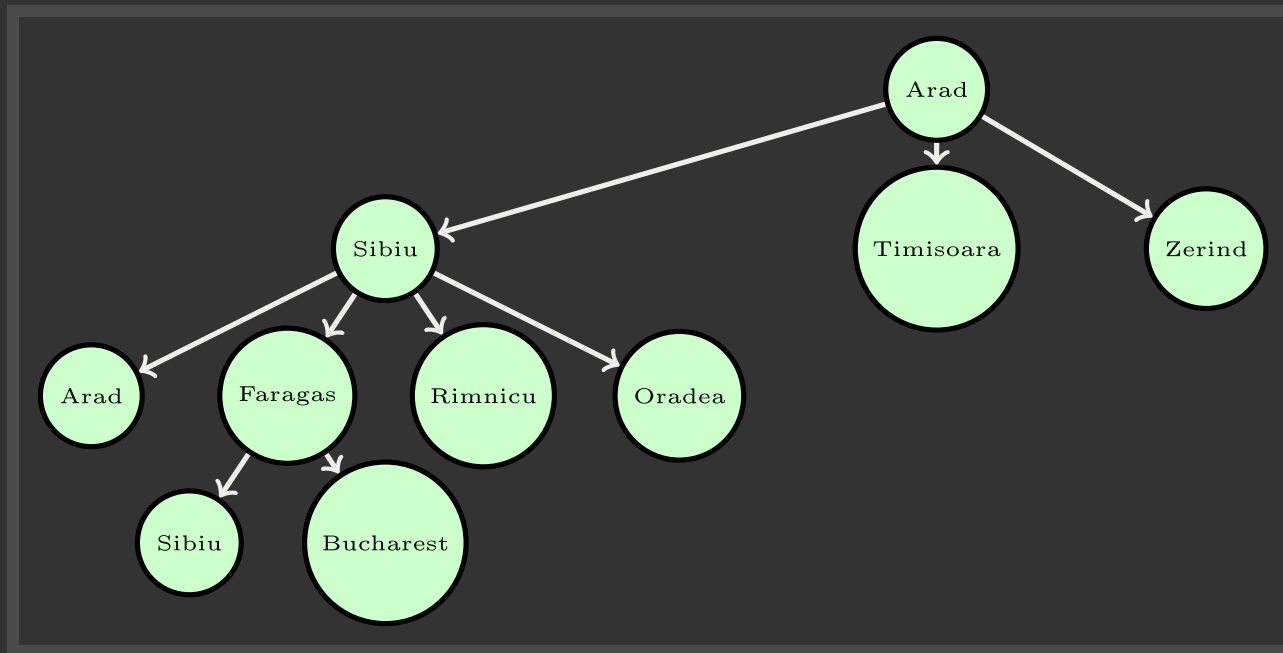
Expand the node that seems closest...



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

GREEDY SEARCH: ROMANIA

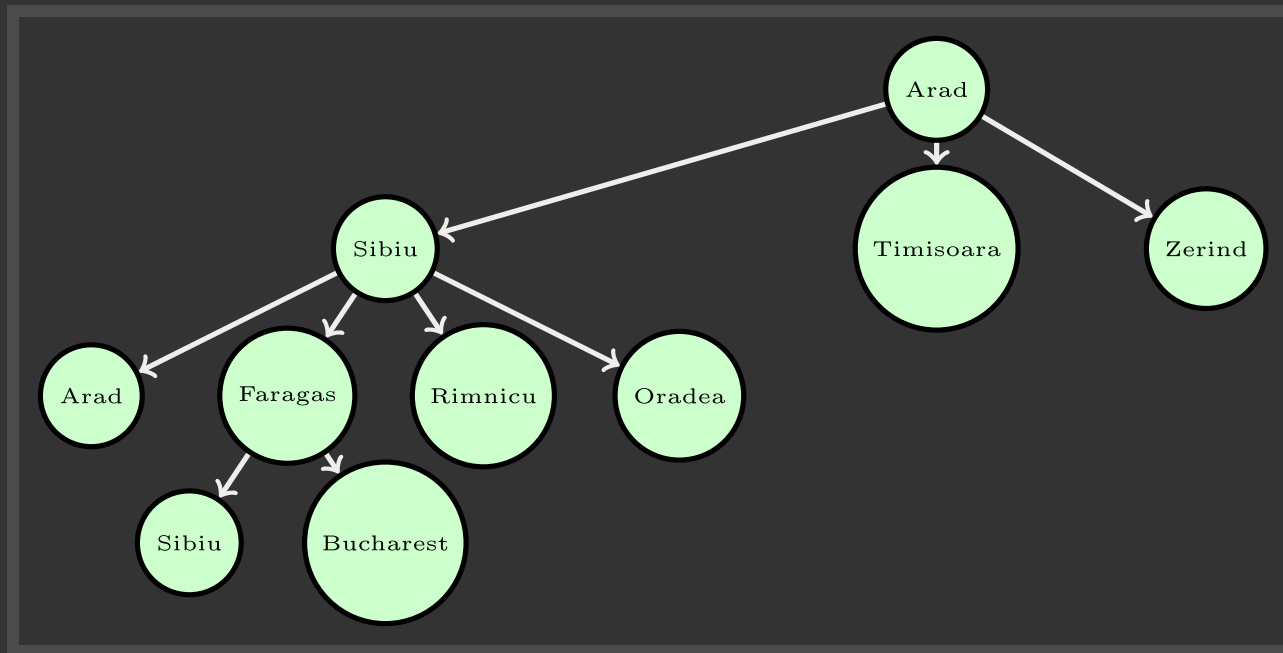
Expand the node that seems closest...



Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

GREEDY SEARCH: ROMANIA

Expand the node that seems closest...



What can go wrong?

Straight-line distance to Bucharest	
Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

GREEDY SEARCH

Strategy: expand a node that you think is closest to a goal state

- Heuristic: distance estimate to nearest goal for each state

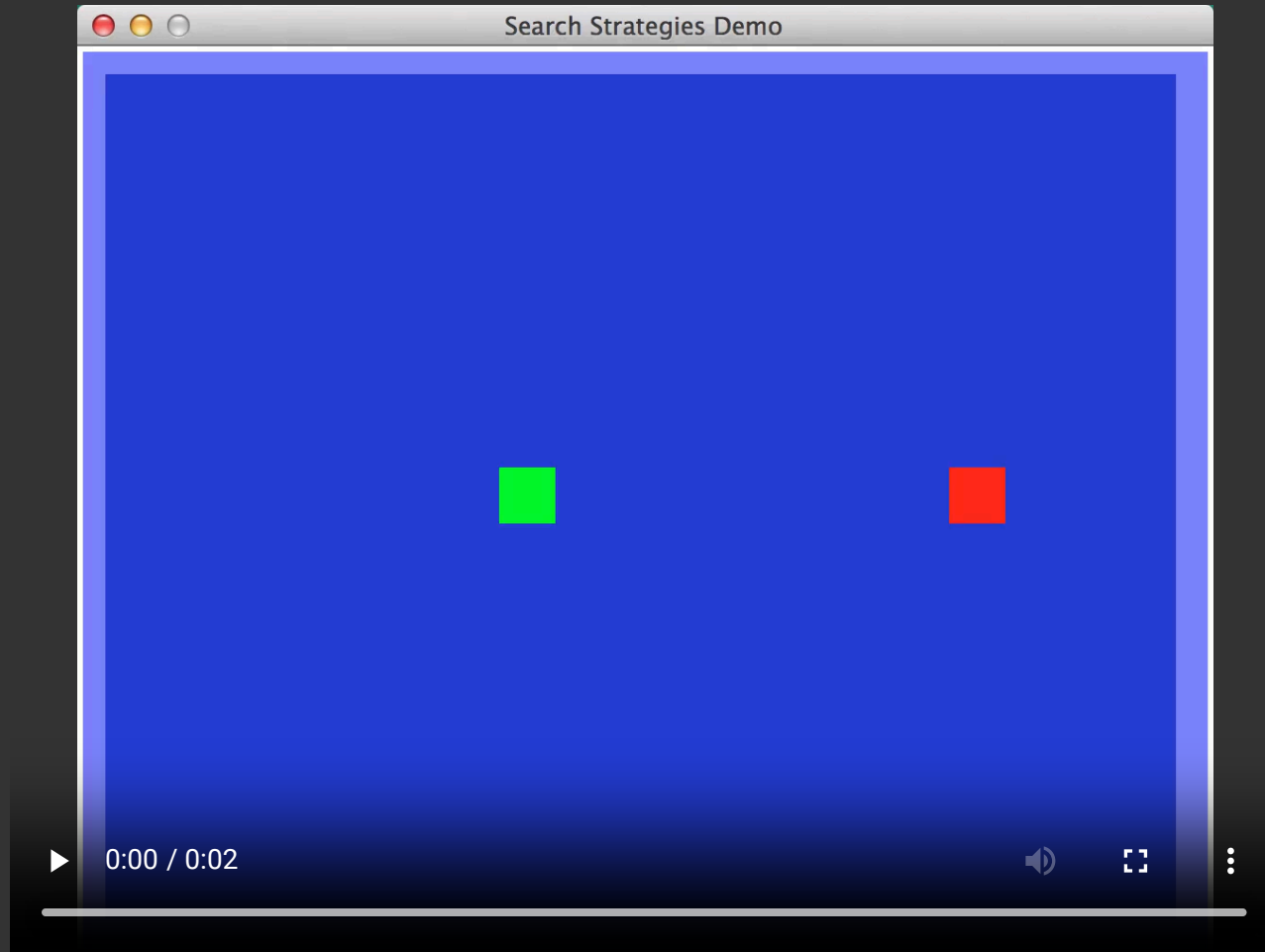
A common case:

- Best-first takes you straight to the (wrong) goal

Worst-case: like a badly-guided DFS



GREEDY DEMO

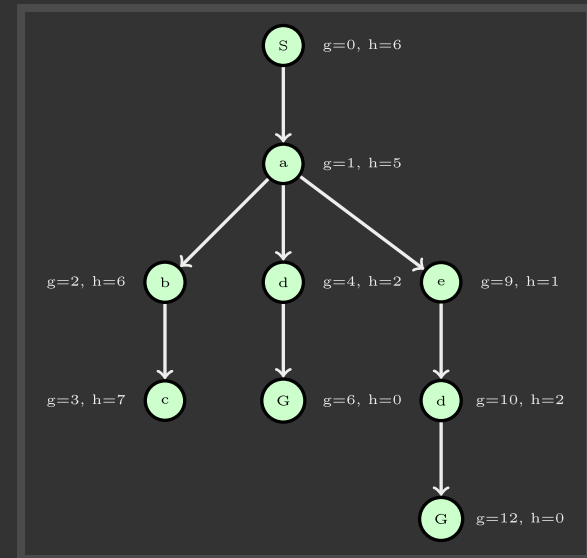
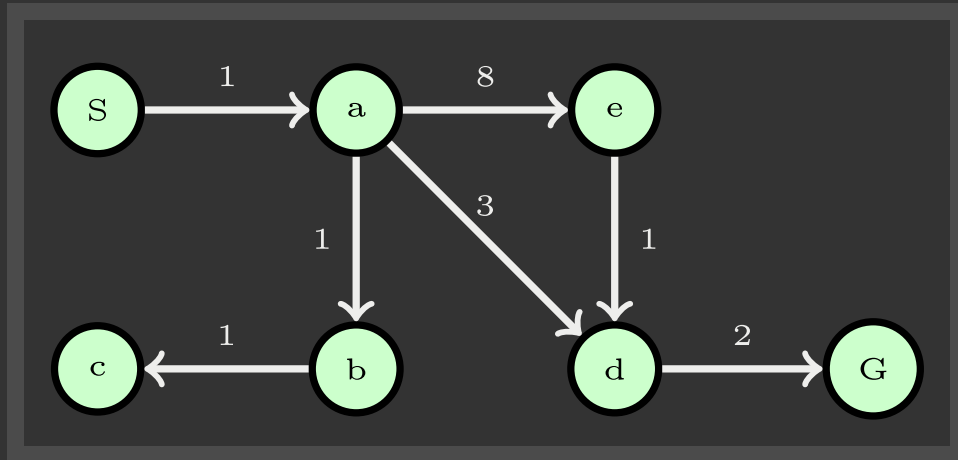


A^* SEARCH

KEY IDEA: COMBINE UCS AND GREEDY

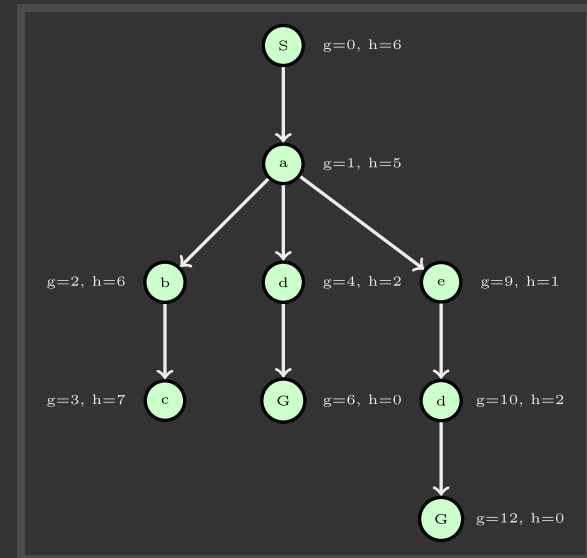
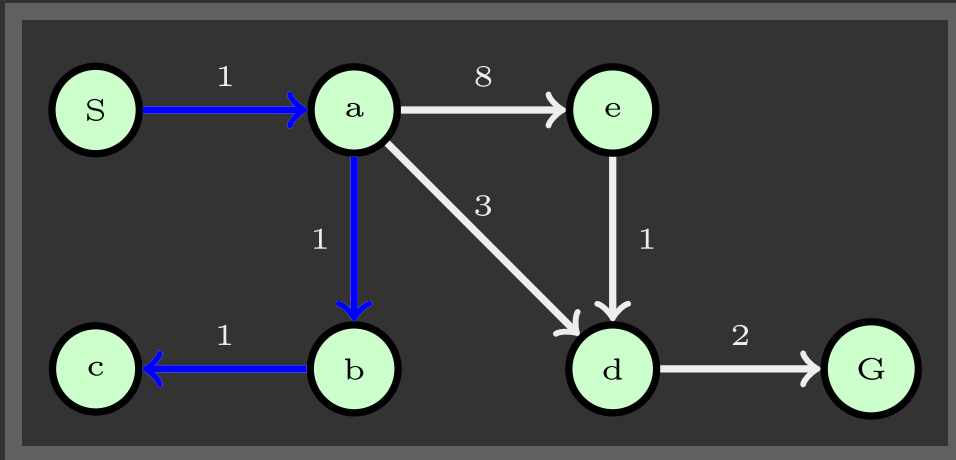


COMBINING UCS AND GREEDY



COMBINING UCS AND GREEDY

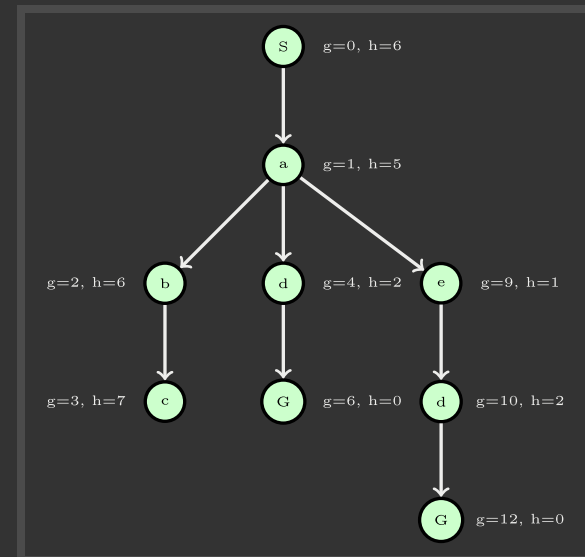
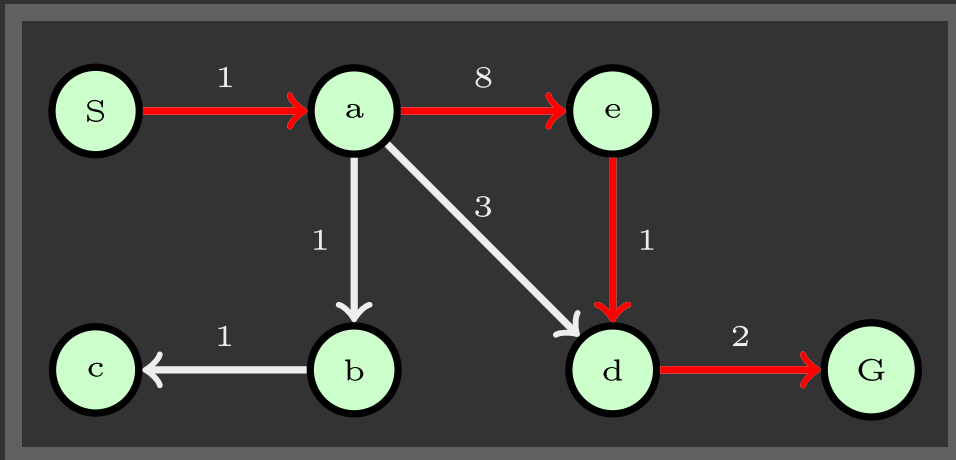
Uniform-cost orders by path cost, or backward cost $g(n)$.



COMBINING UCS AND GREEDY

Uniform-cost orders by path cost, or backward cost $g(n)$.

Greedy orders by goal proximity, or forward cost $h(n)$.

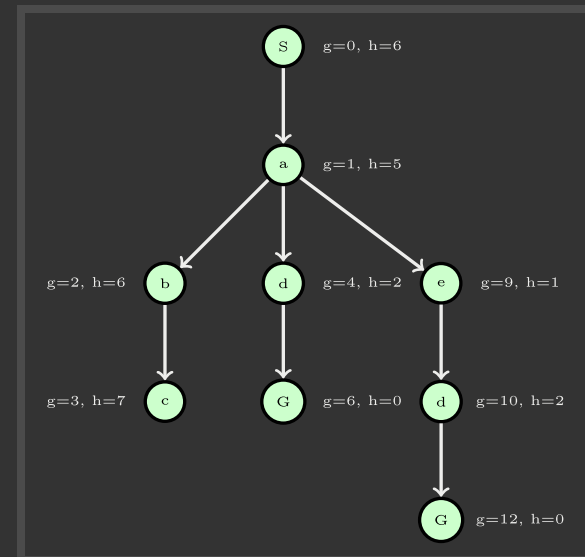
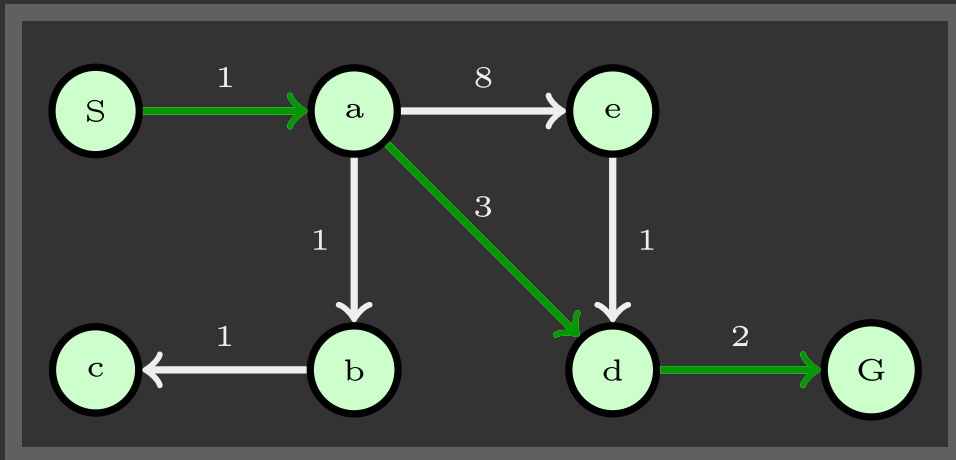


COMBINING UCS AND GREEDY

Uniform-cost orders by path cost, or backward cost $g(n)$.

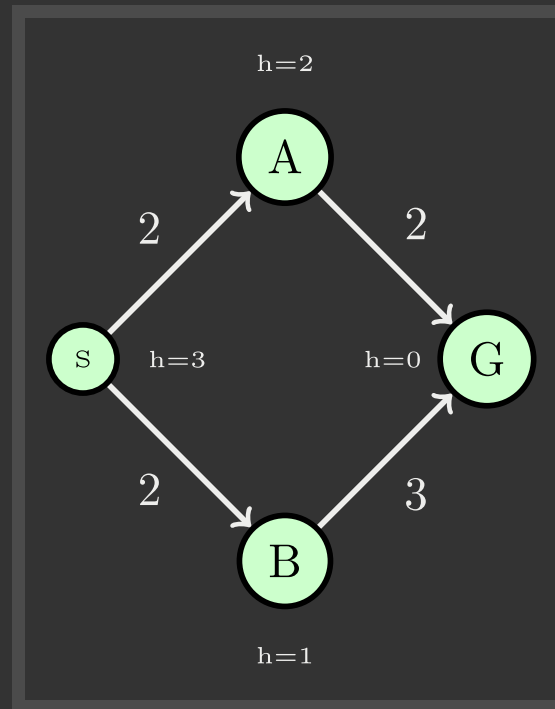
Greedy orders by goal proximity, or forward cost $h(n)$.

A^* Search orders by the sum: $f(n) = g(n) + h(n)$



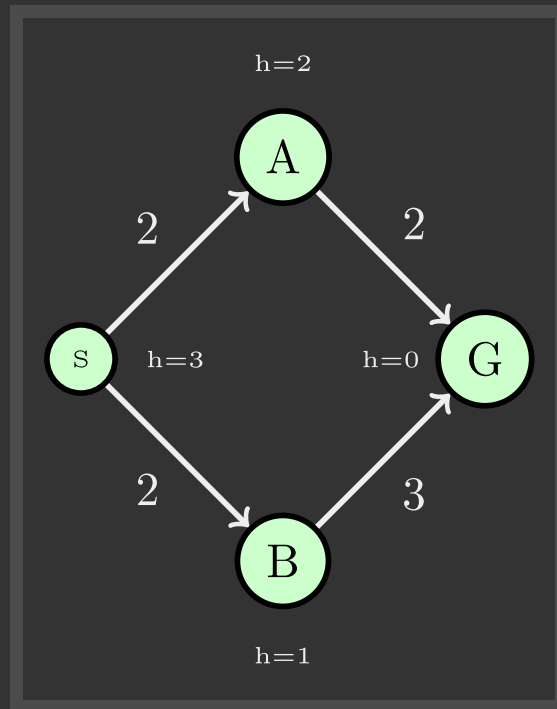
WHEN SHOULD A^* TERMINATE?

Should we stop when we enqueue a goal?



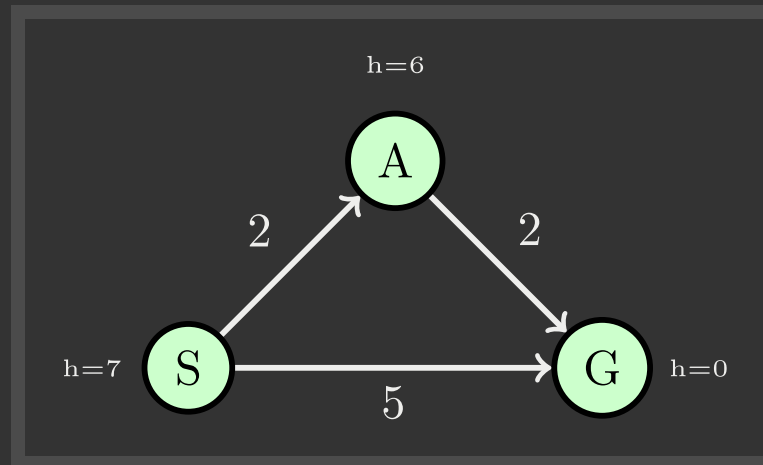
WHEN SHOULD A^* TERMINATE?

Should we stop when we enqueue a goal?

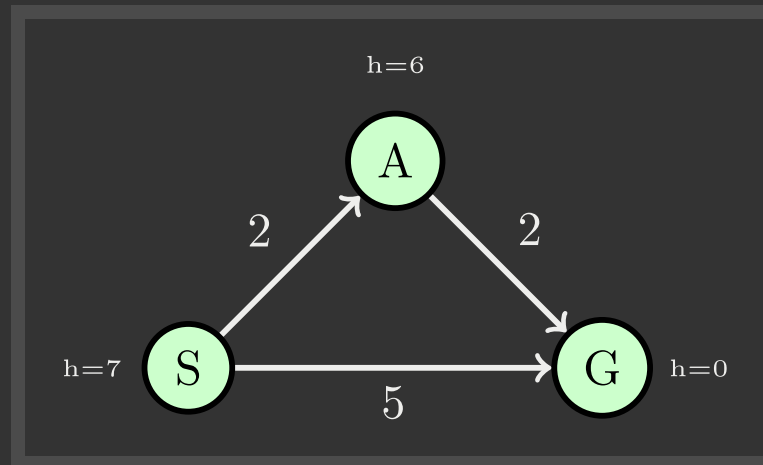


No: only stop when we dequeue a goal

IS A^* OPTIMAL?

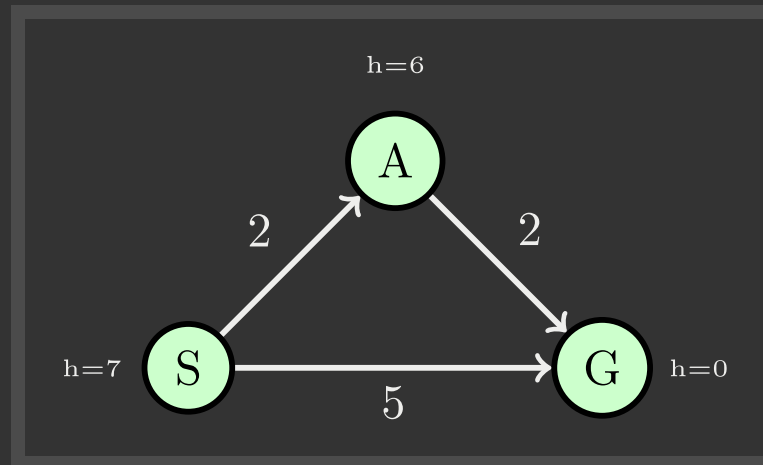


IS A^* OPTIMAL?



What went wrong?

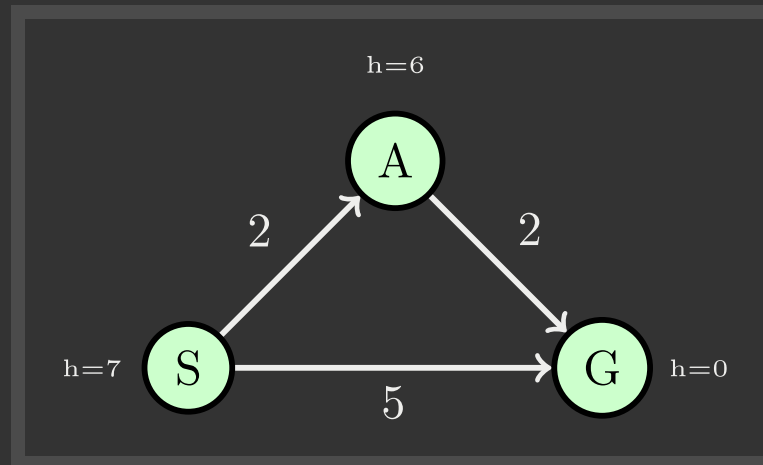
IS A^* OPTIMAL?



What went wrong?

Actual goal cost $\leq$ estimated goal cost

IS A^* OPTIMAL?



What went wrong?

Actual goal cost $\leq$ estimated goal cost

We need estimates to be less than actual costs

ADMISSIBILITY

Inadmissible (pessimistic) heuristics break optimality by trapping good plans on the fringe.

Admissible (optimistic) heuristics slow down bad plans but never outweigh true costs



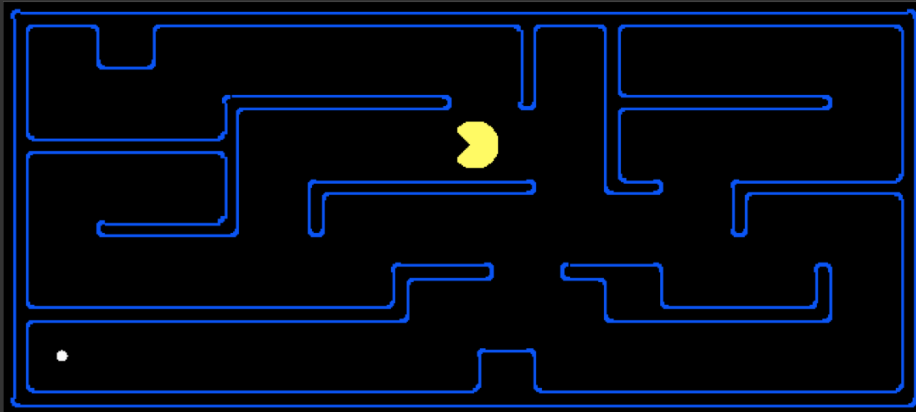
ADMISSIBLE HEURISTICS

A heuristic h is admissible (optimistic) if:

$$0 \leq h(n) \leq h^*(n)$$

where $h^*(n)$ is the true cost to a nearest goal

Example:



In practice, coming up with an admissible heuristic is most of what is involved in using A^* .



OPTIMALITY OF A^* TREE SEARCH

Assume:

- A is an optimal goal node
- B is a suboptimal goal node
- h is admissible

Claim:

- A will exit the fringe before B



OPTIMALITY OF A^* TREE SEARCH

Proof:



OPTIMALITY OF A^* TREE SEARCH

Proof:

Imagine B is on the fringe

Some ancestor n of A is on the fringe too



OPTIMALITY OF A^* TREE SEARCH

Proof:

Imagine B is on the fringe

Some ancestor n of A is on the fringe too

Claim: n will be expanded before B



OPTIMALITY OF A^* TREE SEARCH

Proof:

Imagine B is on the fringe

Some ancestor n of A is on the fringe too

Claim: n will be expanded before B

- $f(n)$ is less or equal to $f(A)$

$$f(n) = g(n) + h(n)$$

$$f(n) \leq g(A) \text{ admissibility of } A$$

$$g(A) = f(A) \quad h = 0 \text{ at goal}$$



OPTIMALITY OF A^* TREE SEARCH

Proof:

Imagine B is on the fringe

Some ancestor n of A is on the fringe too

Claim: n will be expanded before B

- $f(n)$ is less or equal to $f(A)$
- $f(A)$ is less than $f(B)$

$$f(n) = g(n) + h(n)$$

$$f(n) \leq g(A) \text{ admissibility of } A$$

$$g(A) = f(A) \quad h = 0 \text{ at goal}$$

$$g(A) \leq g(B) \quad B \text{ is suboptimal}$$

$$f(A) \leq f(B) \quad h = 0 \text{ at goal}$$



OPTIMALITY OF A^* TREE SEARCH

Proof:

Imagine B is on the fringe

Some ancestor n of A is on the fringe too

Claim: n will be expanded before B

- $f(n)$ is less or equal to $f(A)$
- $f(A)$ is less than $f(B)$
- n expands before B

$$f(n) = g(n) + h(n)$$

$$f(n) \leq g(A) \text{ admissibility of } A$$

$$g(A) = f(A) \quad h = 0 \text{ at goal}$$

$$g(A) \leq g(B) \quad B \text{ is suboptimal}$$

$$f(A) \leq f(B) \quad h = 0 \text{ at goal}$$

$$f(n) \leq f(A) < f(B)$$

OPTIMALITY OF A^* TREE SEARCH

Proof:

Imagine B is on the fringe

Some ancestor n of A is on the fringe too

Claim: n will be expanded before B

- $f(n)$ is less or equal to $f(A)$
- $f(A)$ is less than $f(B)$
- n expands before B

All ancestors of A expand before B

$$f(n) = g(n) + h(n)$$

$$f(n) \leq g(A) \text{ admissibility of } A$$

$$g(A) = f(A) \quad h = 0 \text{ at goal}$$

$$g(A) \leq g(B) \quad B \text{ is suboptimal}$$

$$f(A) \leq f(B) \quad h = 0 \text{ at goal}$$

$$f(n) \leq f(A) < f(B)$$

OPTIMALITY OF A^* TREE SEARCH

Proof:

Imagine B is on the fringe

Some ancestor n of A is on the fringe too

Claim: n will be expanded before B

- $f(n)$ is less or equal to $f(A)$
- $f(A)$ is less than $f(B)$
- n expands before B

All ancestors of A expand before B

A expands before B

$$f(n) = g(n) + h(n)$$

$$f(n) \leq g(A) \text{ admissibility of } A$$

$$g(A) = f(A) \quad h = 0 \text{ at goal}$$

$$g(A) \leq g(B) \quad B \text{ is suboptimal}$$

$$f(A) \leq f(B) \quad h = 0 \text{ at goal}$$

$$f(n) \leq f(A) < f(B)$$



OPTIMALITY OF A^* TREE SEARCH

Proof:

Imagine B is on the fringe

Some ancestor n of A is on the fringe too

Claim: n will be expanded before B

- $f(n)$ is less or equal to $f(A)$
- $f(A)$ is less than $f(B)$
- n expands before B

All ancestors of A expand before B

A expands before B

A^* search is optimal

$$f(n) = g(n) + h(n)$$

$$f(n) \leq g(A) \text{ admissibility of } A$$

$$g(A) = f(A) \quad h = 0 \text{ at goal}$$

$$g(A) \leq g(B) \quad B \text{ is suboptimal}$$

$$f(A) \leq f(B) \quad h = 0 \text{ at goal}$$

$$f(n) \leq f(A) < f(B)$$



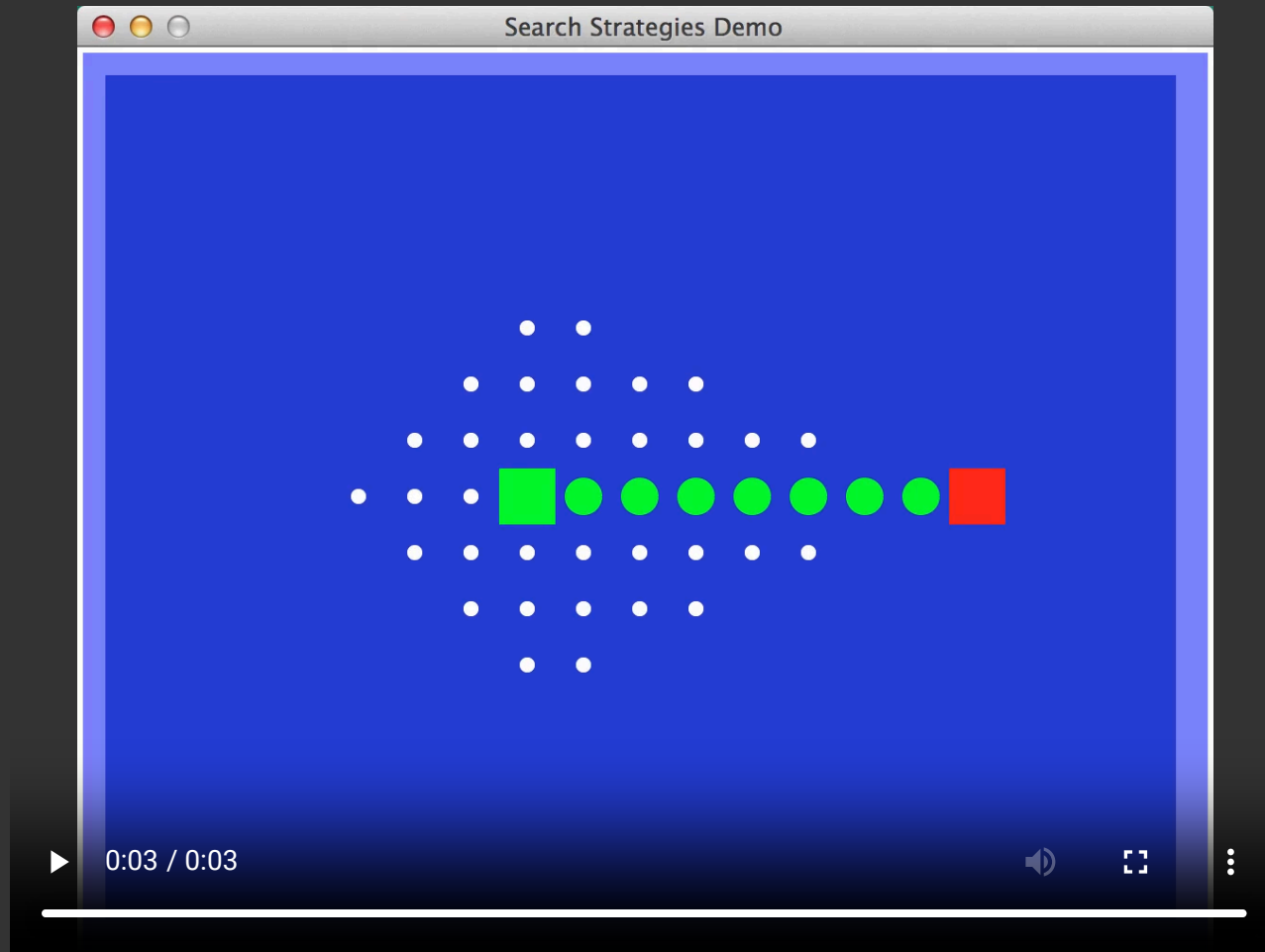
UCS VS A^* CONTOURS

Uniform-cost expands equally in all "directions"

A^* expands mainly toward the goal, but does hedge its bets to ensure optimality



A* DEMO



A* APPLICATIONS

Video games

Pathing / routing problems

Resource planning problems

Robot motion planning

Language analysis

Machine translation

Speech recognition

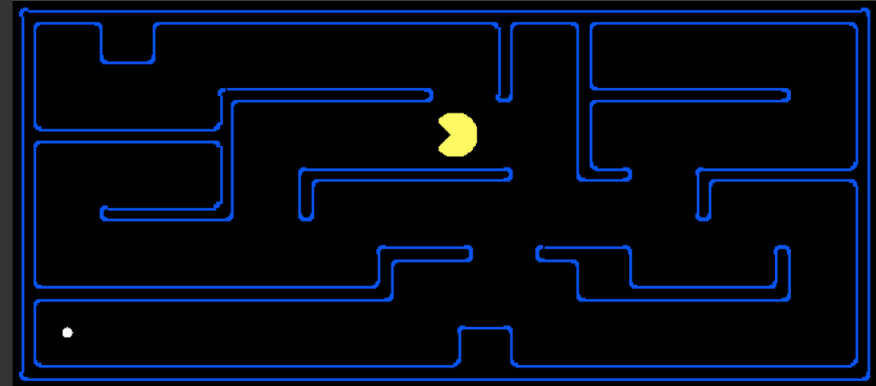
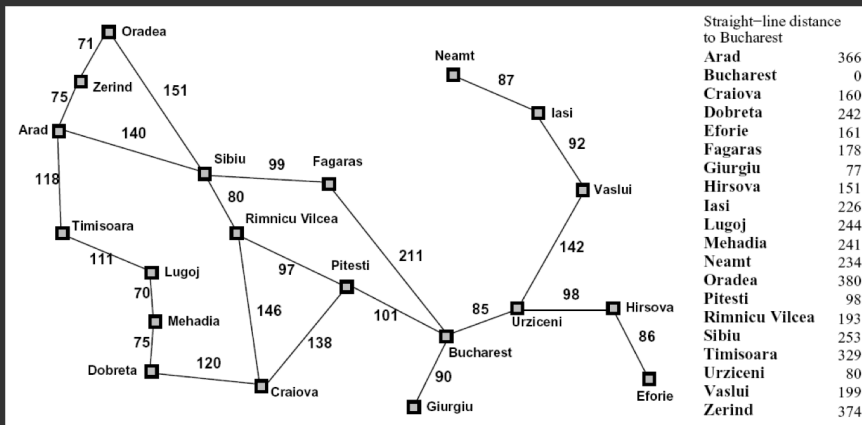
...



CREATING ADMISSIBLE HEURISTICS

Most of the work in solving hard search problems optimally is in coming up with admissible heuristics

Often, admissible heuristics are solutions to relaxed problems, where new actions are available



Inadmissible heuristics can often be useful too.

A^* SUMMARY

A^* uses both backward costs and (estimates of) forward costs

A^* is optimal with admissible heuristics

Heuristic design is key: often use relaxed problems



DESIGNING HEURISTIC AS A SEARCH PROBLEM

nature

View all journalsSearchLog in

Explore content>About the journalPublish with usSign up for alertsRSS feed

[nature](#) > [articles](#) > [article](#)

Article | [Open access](#) | Published: 14 December 2023

Mathematical discoveries from program search with large language models

[Bernardino Romera-Paredes](#)  [Mohammadamin Barekatain](#), [Alexander Novikov](#), [Matej Balog](#), [M. Pawan Kumar](#), [Emilien Dupont](#), [Francisco J. R. Ruiz](#), [Jordan S. Ellenberg](#), [Pengming Wang](#), [Omar Fawzi](#), [Pushmeet Kohli](#)  & [Alhussein Fawzi](#) 

[Nature](#) **625**, 468–475 (2024) | [Cite this article](#)

148k Accesses | 1 Citations | 988 Altmetric | [Metrics](#)

Abstract

Large language models (LLMs) have demonstrated tremendous capabilities in solving complex tasks, from quantitative reasoning to understanding natural language. However, LLMs sometimes suffer from confabulations (or hallucinations), which can result in them making plausible but incorrect statements^{1,2}. This hinders the use of current large models in scientific discovery. Here we introduce FunSearch (short for searching in the function space), an evolutionary procedure based on pairing a pretrained LLM with a systematic evaluator. We demonstrate the effectiveness of this approach to surpass the best-known results in important problems, pushing the boundary of existing LLM-based approaches³. Applying FunSearch to a central problem in extremal combinatorics—the cap set problem—we discover new constructions of large cap sets going beyond the best-known ones, both in finite dimensional and asymptotic cases. This shows that it is possible to make discoveries for established open problems using LLMs. We showcase the generality of FunSearch by applying

[Download PDF](#)

Associated content

[Large language models help computer programs to evolve](#)

Jean-Baptiste Mouret
Nature | **News & Views** | 17 Jan 2024

Sections | **Figures** | **References**

[Abstract](#)

[Main](#)

[FunSearch](#)

[Extremal combinatorics](#)

[Bin packing](#)

[Discussion](#)

[Methods](#)

[Data availability](#)

[Code availability](#)

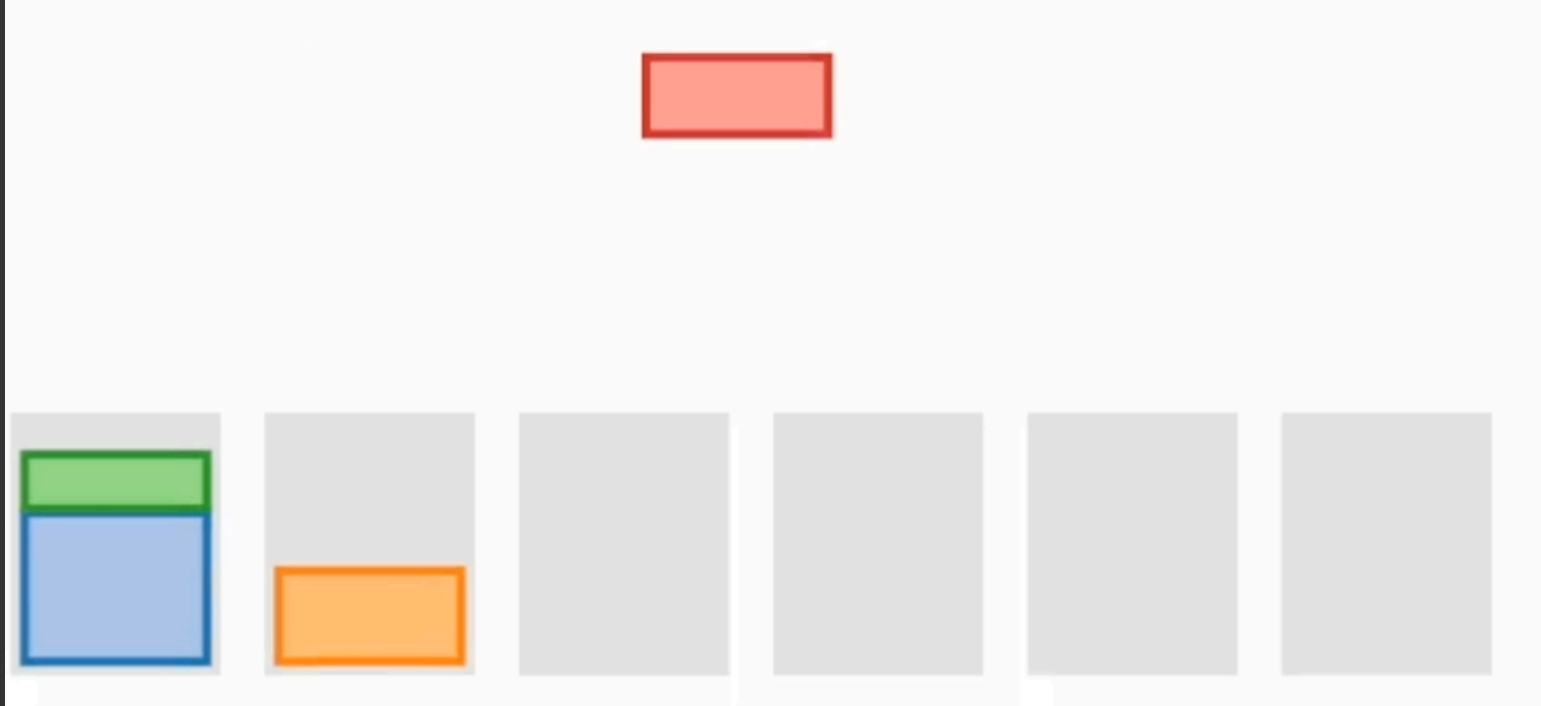
[References](#)

[Acknowledgements](#)

[Author information](#)

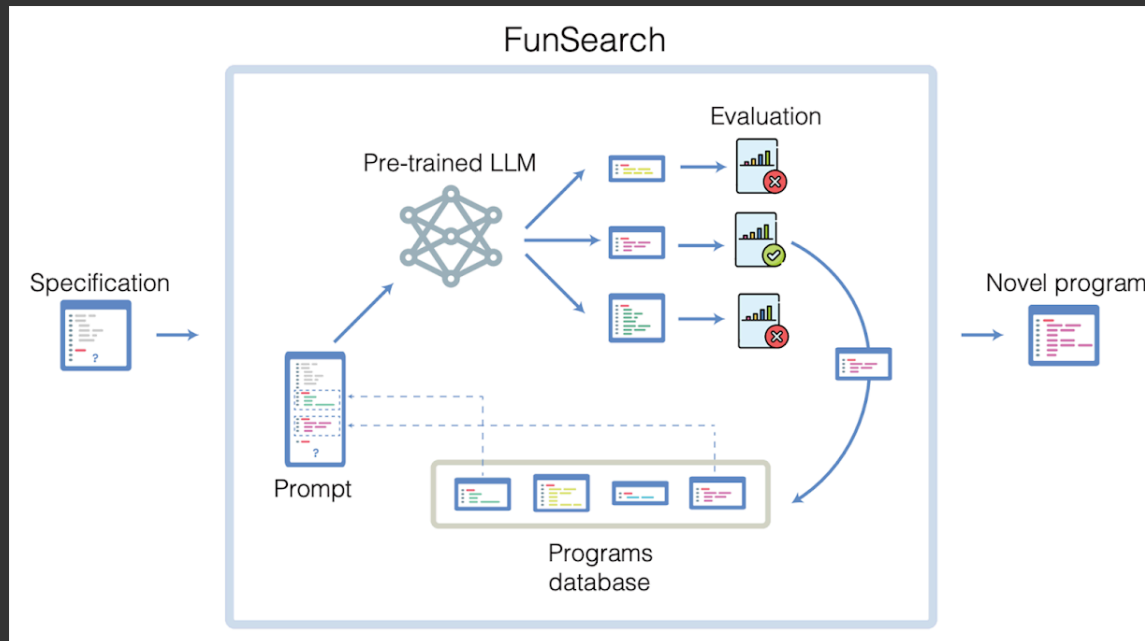
DESIGNING HEURISTIC AS A SEARCH PROBLEM

ONLINE BIN PACKING



DESIGNING HEURISTIC AS A SEARCH PROBLEM

ONLINE BIN PACKING



DESIGNING HEURISTIC AS A SEARCH PROBLEM

ONLINE BIN PACKING

```
def best_fit_heuristic(item: float, bins: np.ndarray) -> np.ndarray:
    """Returns priority with which we want to add item to each bin.

    Args:
        item: Size of item to be added to the bin.
        bins: Array of capacities for each bin.

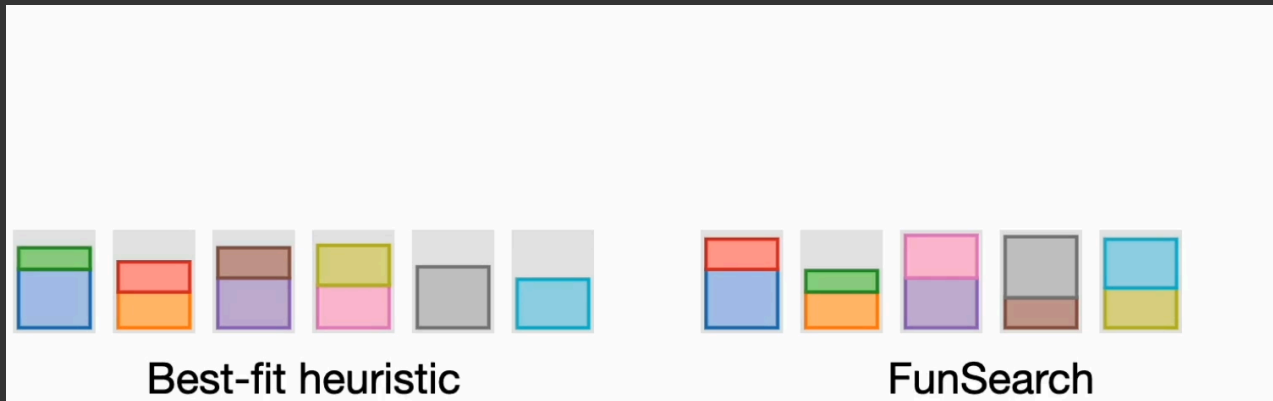
    Return:
        Array of same size as bins with priority score of each bin.
    """
    return np.argmax(bins - item)
```

```
def FunSearch_heuristic(item: float, bins: np.ndarray) -> np.ndarray:
    """Heuristic discovered for the Weibull datasets."""
    max_bin_cap = max(bins)
    score = (bins - max_bin_cap)**2 / item + bins**2 / (item**2)
    score += bins**2 / item**3
    score[bins > item] = -score[bins > item]
    score[1:] -= score[:-1]
    return np.argmax(score)
```



DESIGNING HEURISTIC AS A SEARCH PROBLEM

ONLINE BIN PACKING



▶ 0:13 / 0:13



Q & A



XKCD

